

Bheeshma L
192511332

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks:100

Annexure A

Assignment

Code of Conduct:

I Bheeshma L / 19251332 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Bheeshma L

Course Code & Name **CSA11 – Object Oriented Analysis and Design**

Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS)
Course Outcome(s) – CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS). The system shall integrate patients, doctors, reception staff, administrators, laboratory personnel, and pharmacy staff through a centralized intelligent platform. Patients shall register, manage profiles, search doctors, book/reschedule/cancel appointments, receive reminders, view prescriptions and laboratory reports, and submit service requests. Doctors shall manage availability, appointments, clinical notes, prescriptions, and follow-up requests. The system shall provide intelligent doctor/slot recommendations based on specialty, availability, patient preferences, and historical appointment patterns. Laboratory and pharmacy modules shall update report and prescription status. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply appropriate design principles and patterns to achieve modularity, maintainability, reusability, security, and scalability.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
		deployment environment with strong justification.			
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	100				

Table of Contents

1. Problem Analysis and Requirement Summary
2. Actor Identification
3. Use Case Descriptions
4. Use Case Diagram
5. Object and Class Identification
6. Class Diagram
7. Sequence / Interaction Diagram(s)

Bheeshma L
192511332

8. Activity Diagram
9. State Transition Diagram
10. Package Diagram
11. Component Diagram
12. Deployment Diagram
13. Object Responsibility Analysis (CRC)
14. Design Axioms/Principles and Design Pattern Justification
15. Assumptions and Constraints
16. Requirement-to-Use Case/Class Traceability
17. Implementation / Prototype and Results
18. Reflection on Design Decisions and Learning Outcomes
19. GitHub Upload

1. Problem Analysis and Requirement Summary

1.1 Problem Analysis

AI-SHAPS is a centralized, intelligent healthcare platform that connects Patients, Doctors, Reception Staff, Administrators, Laboratory Personnel and Pharmacy Staff. Its core problem is coordination: matching patient demand (symptoms, specialty needs, time preferences) with doctor supply (specialty, availability) while keeping clinical records, laboratory reports and prescriptions synchronized and securely accessible to the right actor at the right time. An AI recommendation capability adds a layer of intelligence that ranks doctors and time-slots using specialty match, real-time availability, patient preference and historical appointment/no-show patterns, which is the key differentiator from a conventional appointment-booking system.

1.2 System Boundary

The system boundary encloses appointment scheduling, patient/doctor profile management, clinical note-taking, prescription issuance, laboratory report tracking, service-request handling, notifications/reminders and the AI recommendation engine. External payment gateways, SMS/email gateways and third-party insurance systems are treated as external actors/systems that the platform integrates with but does not own.

1.3 Functional Requirements Summary

ID	Requirement
FR-01	Patients shall register, log in and manage their profile and medical preferences.
FR-02	Patients shall search for doctors by specialty, availability and rating/preference.
FR-03	Patients shall book, reschedule and cancel appointments.
FR-04	The system shall send automated appointment reminders and notifications.
FR-05	Patients shall view issued prescriptions and laboratory reports online.
FR-06	Patients shall submit general service/support requests.
FR-07	Doctors shall manage their availability calendar and appointment slots.
FR-08	Doctors shall record clinical notes, issue prescriptions and request follow-ups.
FR-09	The system shall recommend doctors/slots using specialty, availability, patient preference and historical patterns (AI engine).
FR-10	Laboratory staff shall upload and update the status of laboratory reports.
FR-11	Pharmacy staff shall update prescription/dispense status.
FR-12	Reception staff shall book/manage appointments on behalf of walk-in patients.
FR-13	Administrators shall manage users, roles and generate operational reports.

1.4 Non-Functional Requirements Summary

- Security: role-based access control, encrypted storage of medical data (HIPAA/PII-style compliance).
- Scalability: platform shall support horizontal scaling of appointment and AI services under peak load.
- Availability: 99.5%+ uptime target; reminder/notification delivery within defined SLA.
- Maintainability: modular, loosely-coupled services following SOLID principles.
- Usability: simple, accessible UI for patients across web and mobile.
- Auditability: all clinical and prescription updates shall be traceable to an actor and timestamp.

2. Actor Identification

Actors represent roles that interact with AI-SHAPS across its boundary. Primary (human) actors initiate use cases; the AI Recommendation Engine is modelled as a supporting system actor that participates in, but does not initiate, use cases.

Actor	Type	Description / Goals
Patient	Primary, Human	Registers, searches doctors, books/reschedules/cancels appointments, views prescriptions & lab reports, receives reminders, raises service requests.
Doctor	Primary, Human	Manages availability, appointments, clinical notes, prescriptions and follow-up requests.
Reception Staff	Primary, Human	Books/manages appointments and registers walk-in patients on behalf of patients.
Administrator	Primary, Human	Manages users/roles, system configuration and generates operational/analytics reports.
Laboratory Personnel	Primary, Human	Uploads and updates the status of laboratory test reports.
Pharmacy Staff	Primary, Human	Updates prescription/dispense status and medicine availability.
AI Recommendation Engine	Supporting, System	Ranks doctors/slots using specialty, availability, preference and historical data; invoked by Patient/Reception use cases.
Notification Gateway (SMS/Email)	Supporting, External System	Delivers reminders and confirmations initiated by the system.
Payment Gateway	Supporting, External System	Processes consultation fee payment (where applicable).

3. Use Case Descriptions

Fully-dressed descriptions are provided for the three most significant use cases; remaining use cases are summarised in the table that follows.

UC-03: Book Appointment

Actors	Patient (primary), Reception Staff (alt.), AI Recommendation Engine (supporting)
Preconditions	Patient/Reception is authenticated; patient profile exists.
Trigger	Patient wants to consult a doctor.
Main Flow	1. Actor searches doctors by specialty/preference. 2. System (AI engine) returns ranked doctor/slot recommendations. 3. Actor selects a doctor and an available slot. 4. System validates slot availability. 5. System creates and confirms the Appointment. 6. System schedules reminder notifications.
Alternate Flow	4a. Slot no longer available -> system shows next-best alternatives (include UC-13).
Postconditions	A confirmed Appointment exists; slot marked booked; confirmation sent.
Exceptions	Payment failure (if fee required) aborts confirmation and releases slot.

UC-13: Get AI Doctor/Slot Recommendation

Actors	AI Recommendation Engine, Patient/Reception (initiator via include)
Preconditions	Search criteria (specialty/date/preference) supplied.
Main Flow	1. Engine fetches doctor availability. 2. Engine fetches patient preference & historical appointment patterns. 3. Engine scores/ranks candidate doctor-slot pairs. 4. Engine returns ranked list to the calling use case.
Postconditions	A ranked recommendation list is returned; no persistent state change.

UC-11: Issue Prescription

Actors	Doctor (primary)
Preconditions	An in-progress/completed appointment exists with clinical notes.
Main Flow	1. Doctor opens the appointment record. 2. Doctor adds/edits clinical notes. 3. Doctor adds medicines to a new Prescription. 4. System persists prescription with status = ISSUED. 5. System notifies patient and pharmacy queue.
Postconditions	Prescription visible to patient (UC-06) and pharmacy (UC-15).

Summary of remaining use cases:

Use Case	Primary Actor	Brief Description
UC-01 Register/Manage Profile	Patient / Reception	Create and maintain patient profile and preferences.
UC-02 Search Doctor	Patient	Search doctors by specialty, availability, rating.
UC-04 Reschedule/Cancel Appointment	Patient / Reception	Change or cancel an existing appointment.
UC-05 Receive Reminders	System / Patient	Automated pre-appointment notifications.
UC-06 View Prescription	Patient	View issued prescriptions and status.
UC-07 View Lab Report	Patient	View laboratory report and status.
UC-08 Submit Service Request	Patient	Raise a non-clinical support/service request.
UC-09 Manage Availability	Doctor	Define/edit available consultation slots.
UC-10 Manage Clinical Notes	Doctor	Record diagnosis/notes for a visit.
UC-12 Request Follow-up	Doctor	Create a follow-up appointment recommendation.
UC-14 Update Lab Report Status	Lab Personnel	Upload report data and change status.
UC-15 Update Prescription/Dispense Status	Pharmacy Staff	Mark prescription dispensed/partially dispensed.
UC-16 Manage Users & Roles	Administrator	Create/deactivate accounts, assign roles.
UC-17 Generate Reports/Analytics	Administrator	Produce operational/utilisation reports.
UC-18 Authenticate/Login	All actors	Credential-based login with role resolution.

4. Use Case Diagram

The diagram below shows all identified actors, the AI-SHAPS system boundary, and the use cases they participate in, including <<include>> and <<extend>> relationships for the AI recommendation flow and the reminder extension of booking.

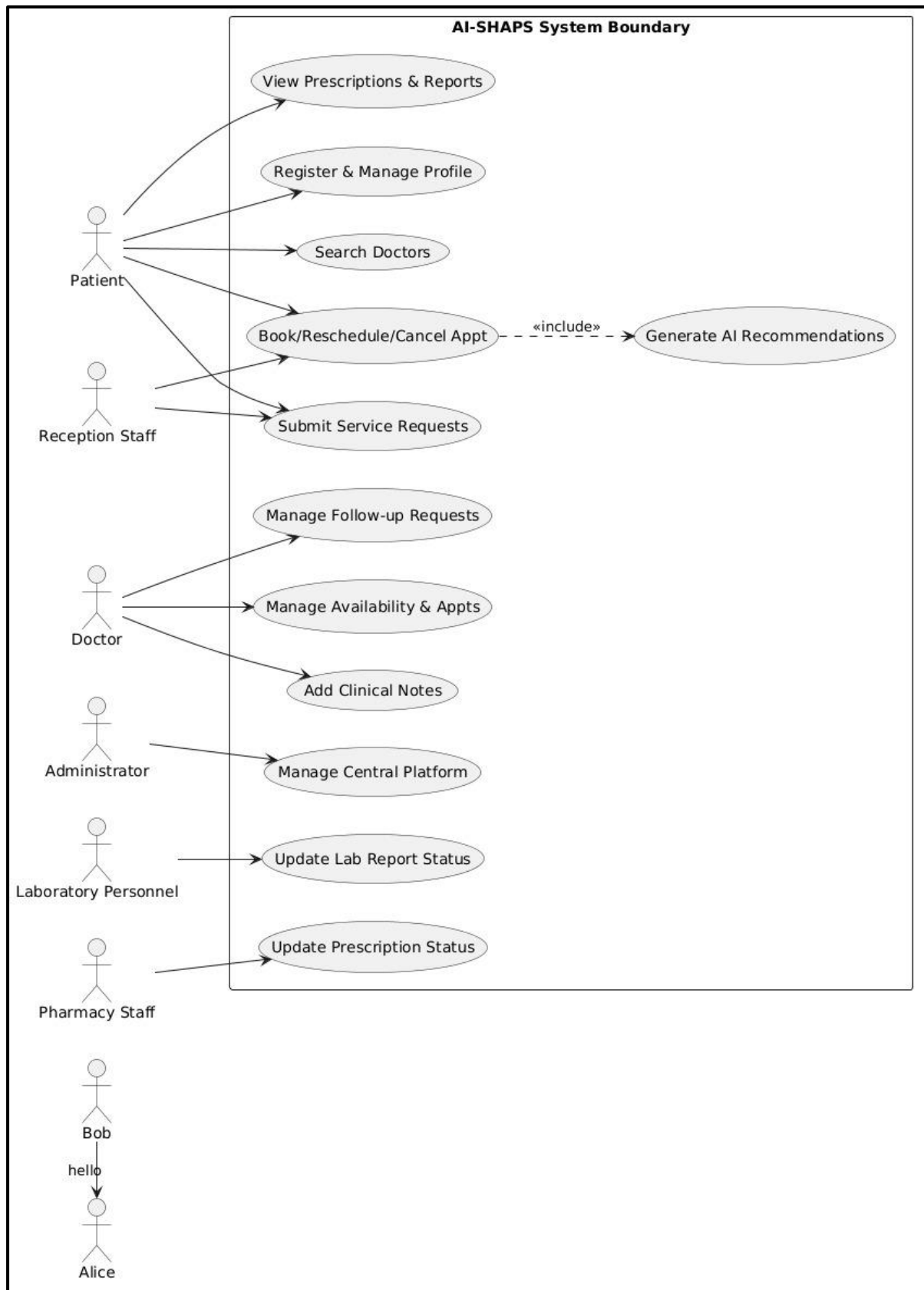


Figure 4.1 - Use Case Diagram (AI-SHAPS)

5. Object and Class Identification

Candidate classes were identified using noun-phrase analysis of the problem statement, then refined by removing duplicates, redundant/attribute-only nouns, and out-of-scope nouns, and by classifying the remainder as Entity, Boundary or Control classes (Jacobson-style).

Class	Stereotype	Justification
Patient, Doctor, ReceptionStaff, Administrator, LabStaff, PharmacyStaff	Entity	Represent the human actors' persistent profile data; specialise the abstract User class.
User (abstract)	Entity	Captures common identity/auth attributes and behaviour shared by all account types (avoids duplication).
Appointment, Slot, ClinicalNote, Prescription, Medicine, LabReport, ServiceRequest, Notification	Entity	Core domain/business objects explicitly named in the problem statement.
AIRecommendationEngine	Control / Service	Encapsulates the intelligent recommendation algorithm; a control object coordinating domain data, not a persistent record.
AppointmentUI, DoctorSearchUI, DashboardUI	Boundary	Represent the interaction surfaces used by actors (implied by 'register', 'search', 'view').
AppointmentService, NotificationService, AuthService	Control	Coordinate use-case logic across entity objects (GRASP Controller).
Role (enumeration)	Type	Constrains valid account roles; not a full class.

Rejected/merged candidate nouns: "platform" (the system itself), "specialty" and "availability" (became attributes of Doctor/Slot rather than classes), "history" (modelled as a query over past Appointment records rather than a separate class).

6. Class Diagram

User is modelled as an abstract superclass to apply inheritance/polymorphism across the six role subclasses. Composition is used where the child cannot exist without the parent (Appointment-Slot binding, Prescription-Medicine); aggregation/association is used for looser ownership (Patient-LabReport, Patient-ServiceRequest). The AIRecommendationEngine is a stereotyped «service» class dependency-linked (<<uses>>) to Appointment rather than tightly coupled to it.

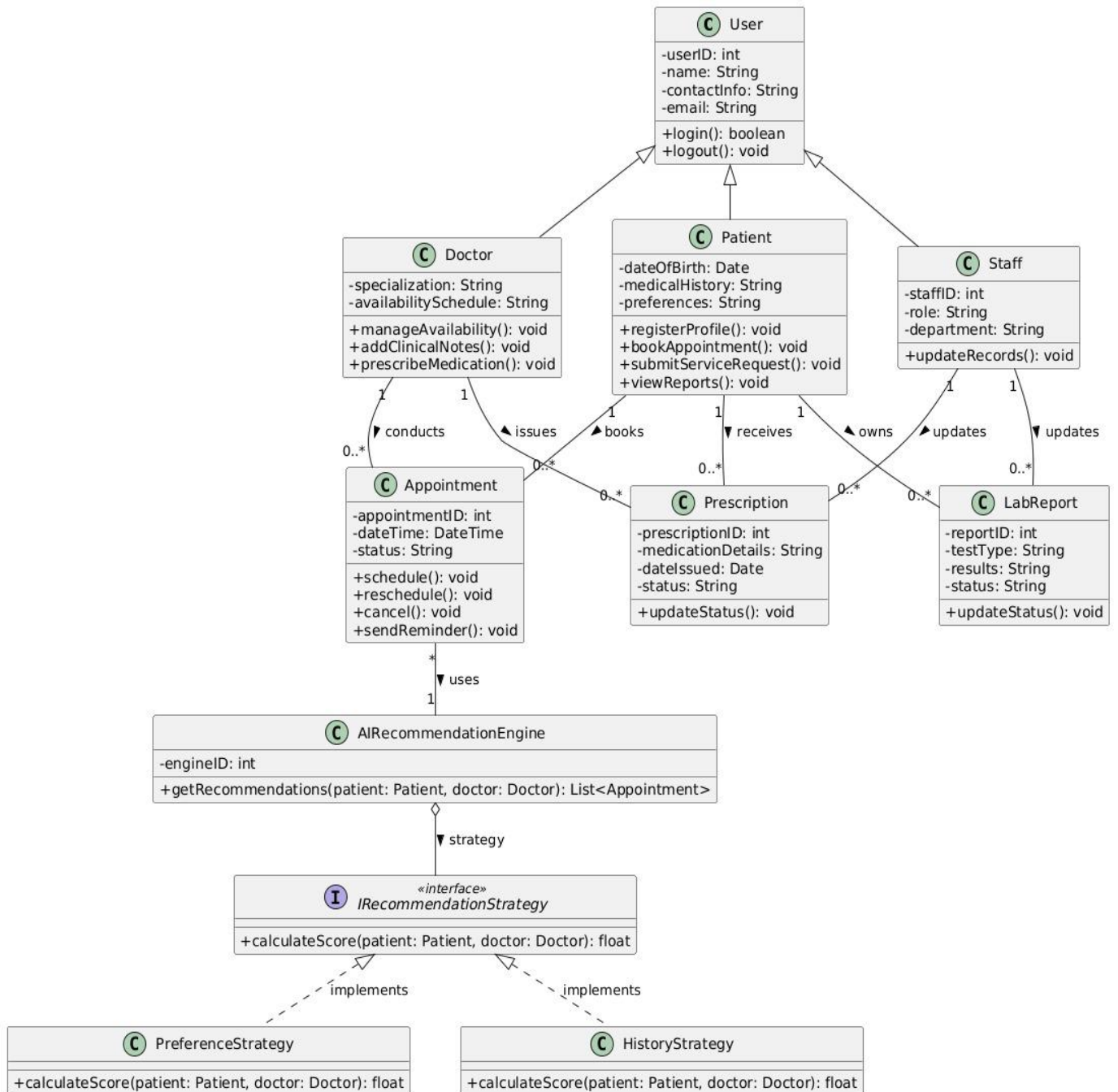


Figure 6.1 - Class Diagram (core domain classes)

7. Sequence / Interaction Diagram(s)

The sequence diagram below traces UC-03 (Book Appointment), including the include-relationship call into UC-13 (AI recommendation). Every message maps to an operation defined on the corresponding class in Section 6, satisfying the class-to-interaction consistency constraint.

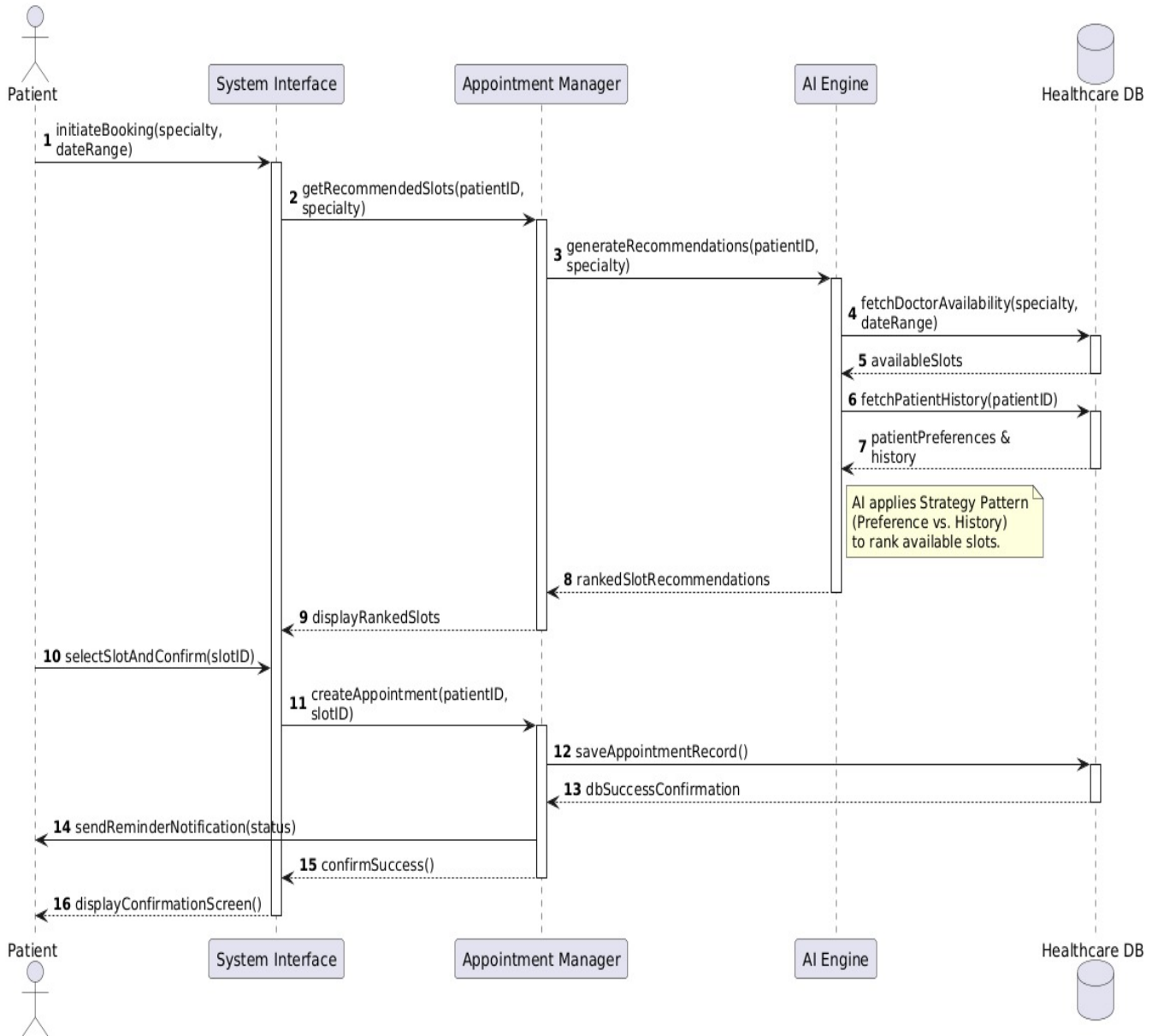


Figure 7.1 - Sequence Diagram: Book Appointment (with AI Recommendation)

Operation traceability for this interaction:

Message	Class.Operation
searchDoctor(criteria)	Patient.searchDoctor(criteria)
recommendDoctor(patient, criteria)	AIRecommendationEngine.recommendDoctor()
bookAppointment(patientId, doctorId, slotId)	Patient.bookAppointment(slot) / AppointmentService
checkAvailability(slotId)	Slot (isBooked check)
persistAppointment()	Appointment.confirm()
sendConfirmation(appointmentId)	Notification.send()

8. Activity Diagram

The activity diagram models the control flow (including the decision branch for slot-conflict handling) underlying the Book Appointment use case, complementing the object-interaction view in Section 7.

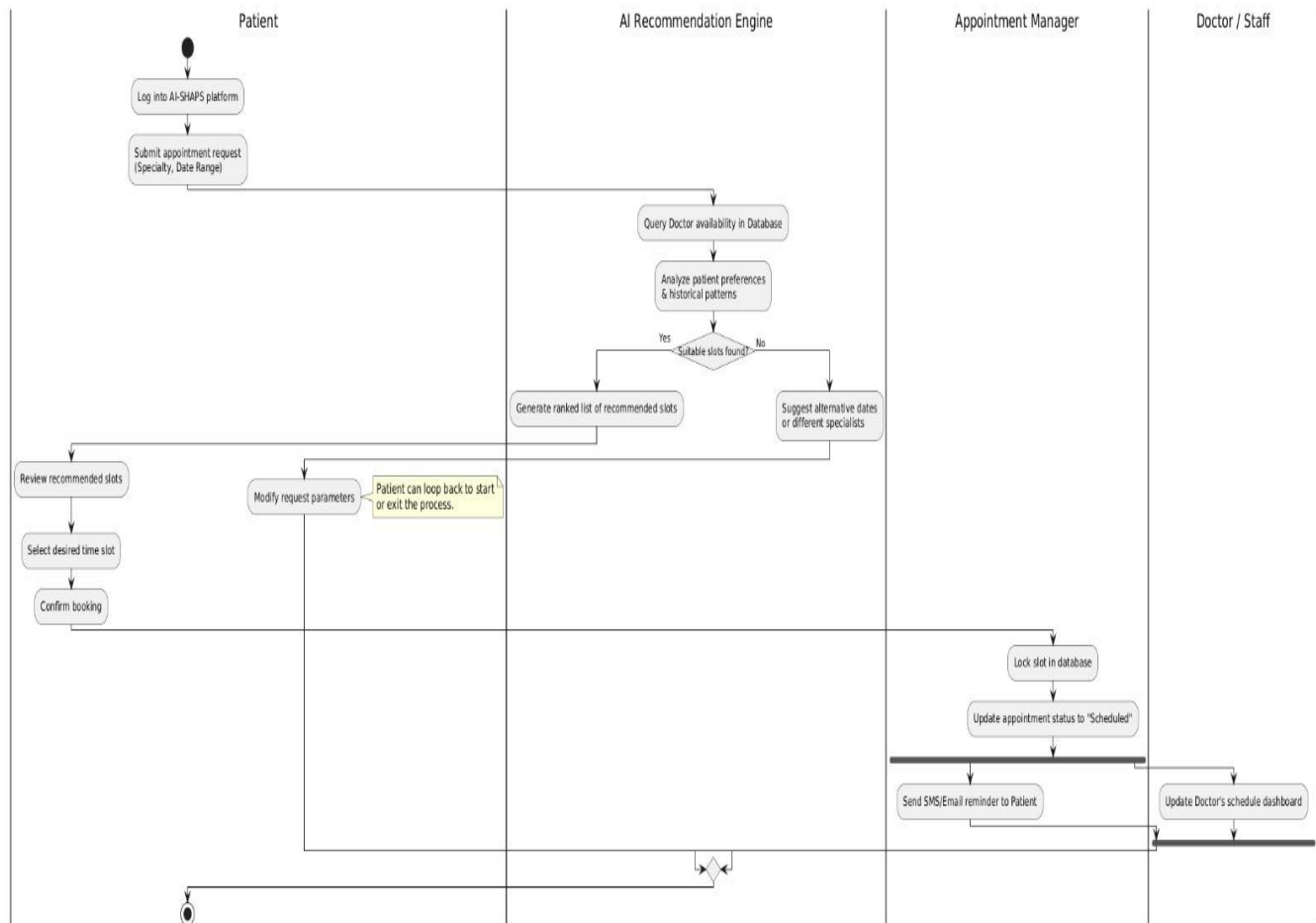
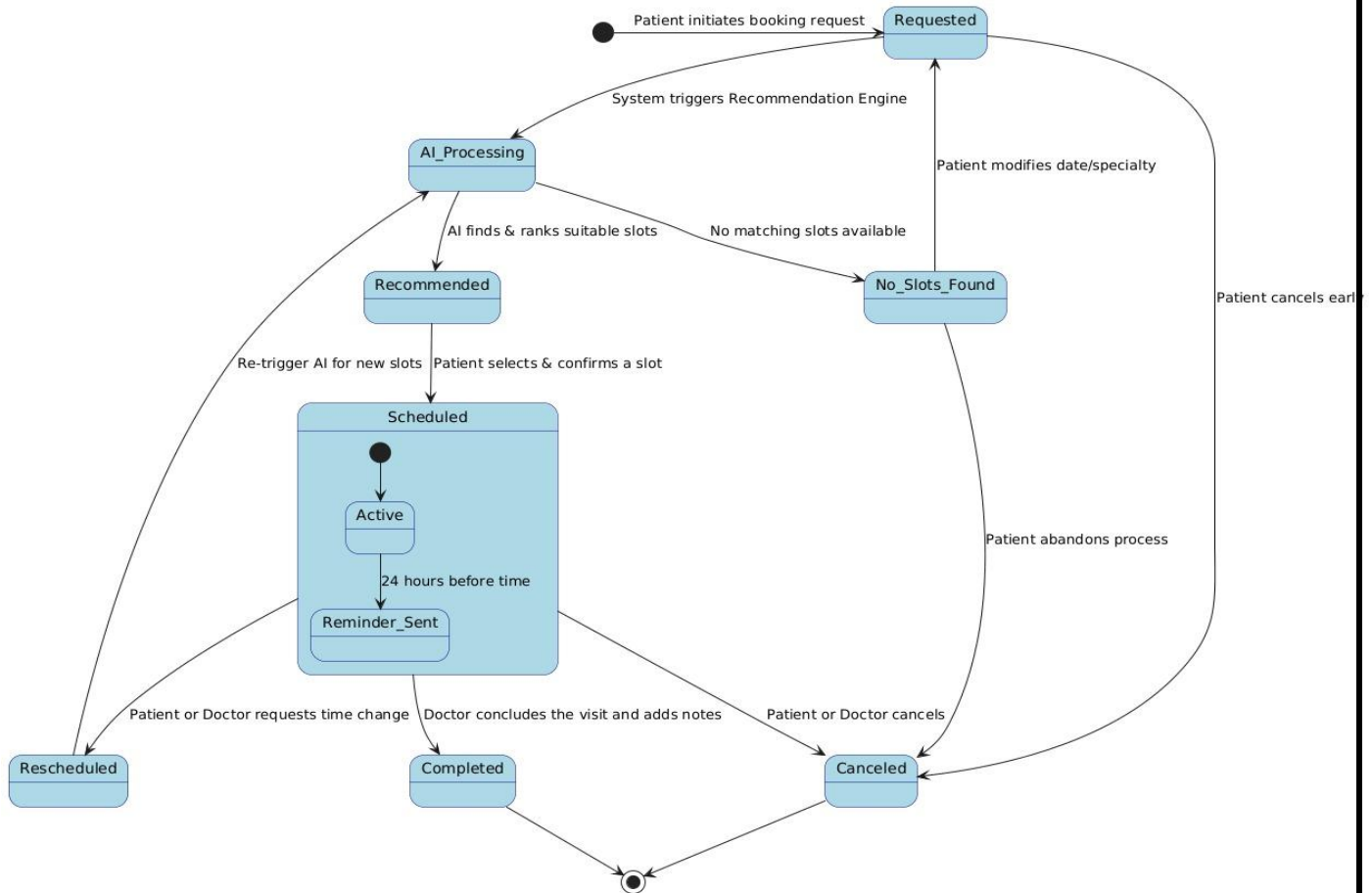


Figure 8.1 - Activity Diagram: Appointment Booking Workflow

9. State Transition Diagram

The Appointment class exhibits the richest lifecycle in the system and is therefore chosen for the state machine. States map directly to the ApptStatus attribute/enumeration on the Appointment class.

Figure 9.1 - State Transition Diagram: Appointment Lifecycle



10. Package Diagram

Classes are grouped into cohesive, loosely-coupled packages following a layered architecture: a Presentation layer, an Application/Controller layer, a Domain layer, and cross-cutting Security, Notification, AI and Persistence packages, so that the AI module and the notification/integration module can be modified or replaced independently of the core domain model.

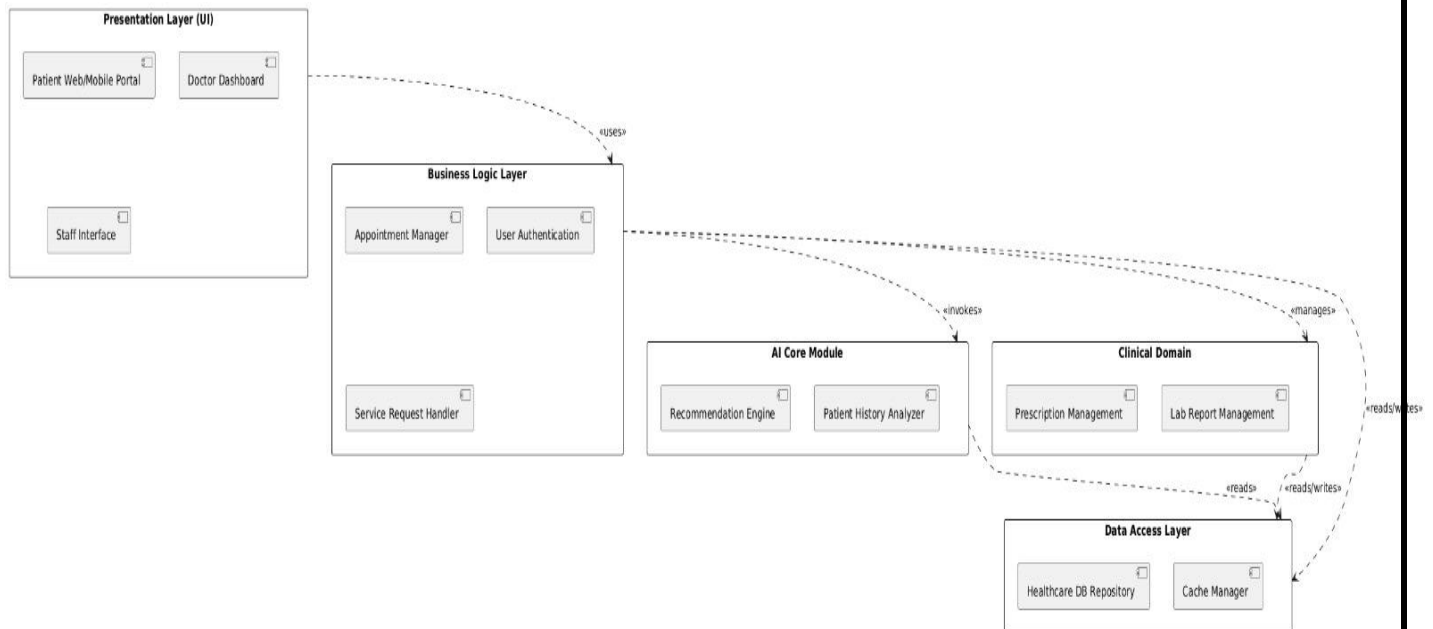


Figure 10.1 - Package Diagram (layered architecture)

11. Component Diagram

At runtime the packages are realised as independently deployable components/services behind an API Gateway, communicating over REST for synchronous calls and a message queue for asynchronous notification delivery -- supporting the scalability and maintainability constraints.

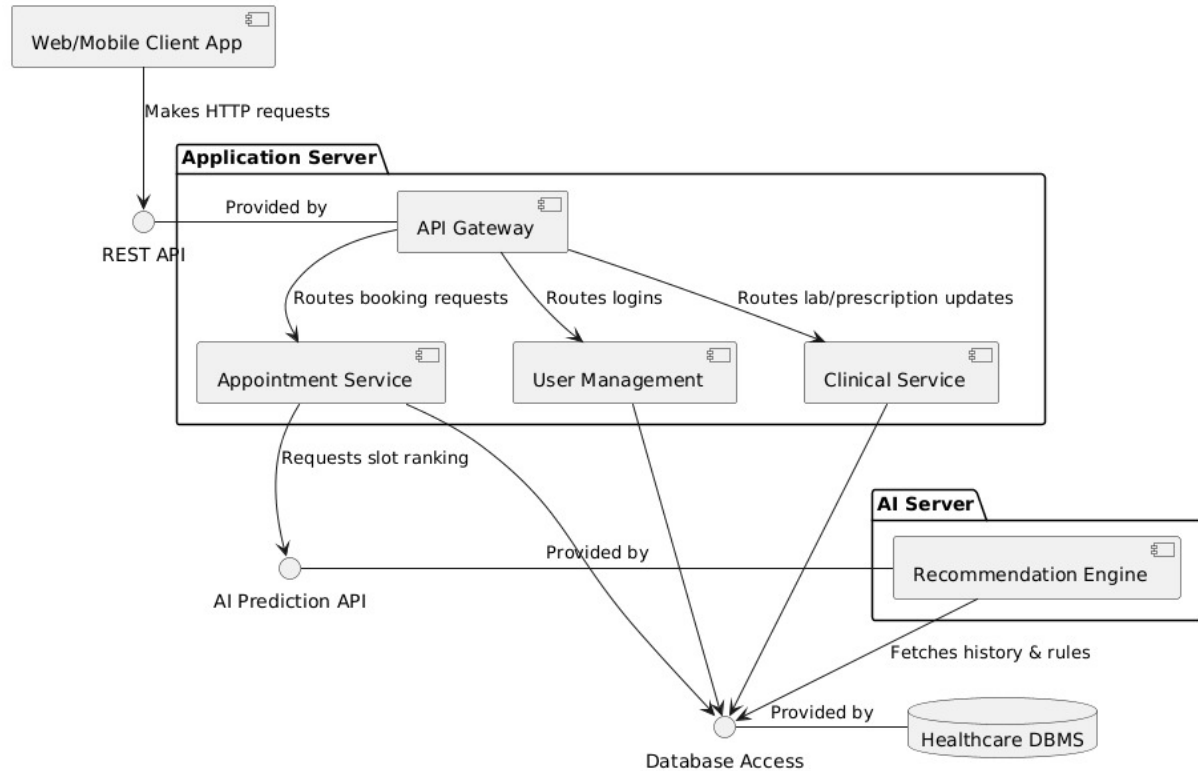
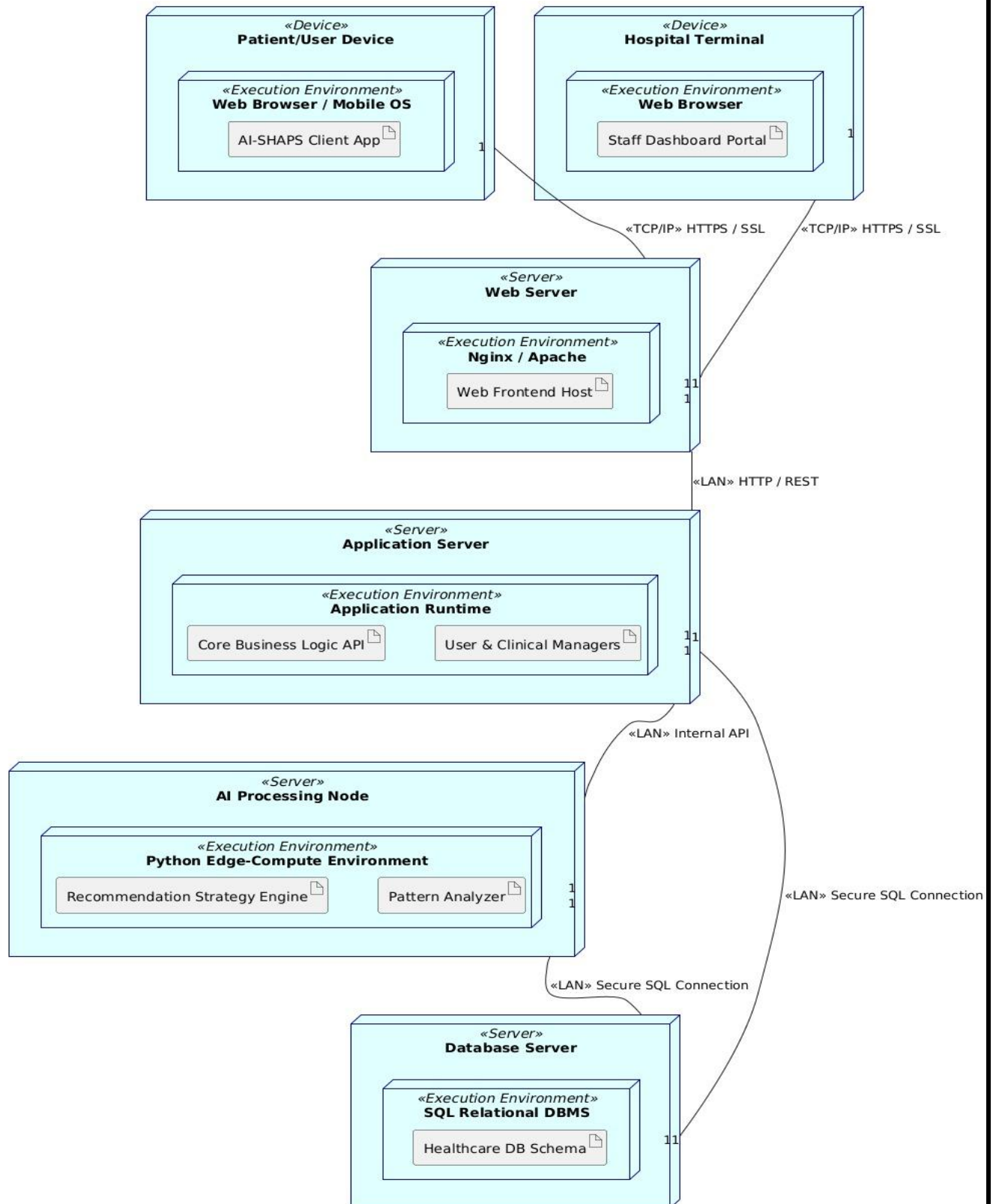


Figure 11.1 - Component Diagram

12. Deployment Diagram

The deployment view shows a horizontally-scaled application-server cluster behind a load balancer, a dedicated AI/ML inference node (so recommendation workloads do not starve transactional traffic), a database server, and an external SMS/Email gateway integration.



13. Object Responsibility Analysis

CRC (Class-Responsibility-Collaborator) analysis was used to assign responsibilities using GRASP principles (Information Expert, Controller, Low Coupling, High Cohesion).

Class	Key Responsibilities	Collaborators
Appointment	Knows its own state and transitions (confirm/reschedule/cancel/complete); enforces valid state changes (Information Expert for its own lifecycle).	Slot, Patient, Doctor, Notification
AIRecommendationEngine	Computes ranked doctor/slot suggestions; isolates the ML/heuristic logic from the domain model (High Cohesion, Low Coupling).	Doctor, Slot, Patient, Appointment (read-only)
AppointmentService (Controller)	Coordinates the multi-object booking use case; delegates to Appointment, Slot, AIRecommendationEngine and Notification rather than doing their work itself (GRASP Controller).	Appointment, Slot, AIRecommendationEngine, NotificationService
Doctor	Owens and validates its own availability (Slot list); authorises clinical actions (notes, prescriptions).	Slot, ClinicalNote, Prescription
Prescription	Knows its own medicines and dispense status; validates status transitions.	Medicine, PharmacyStaff
LabReport	Knows its own result data and status; restricts status change to authorised Lab Personnel.	LabStaff, Patient
NotificationService	Single responsibility for composing and dispatching messages across channels, decoupling business services from delivery mechanics.	Notification, external SMS/Email gateway

14. Design Axioms/Principles and Design Pattern Justification

14.1 Design Axioms and OO Principles Applied

Principle	Application in AI-SHAPS
Encapsulation	Each entity class exposes state only via methods (e.g., Appointment.reschedule() rather than direct field mutation), protecting invariants such as 'a slot cannot be double-booked'.
Abstraction	User is an abstract class exposing login()/logout()/updateProfile() without committing to how each role implements them.
Inheritance	Patient, Doctor, ReceptionStaff, Administrator, LabStaff, PharmacyStaff inherit shared identity/auth behaviour from User, avoiding duplication.
Polymorphism	Role-specific dashboards/permissions are resolved via the common User interface, e.g., a generic AuthService.authorize(User) works uniformly across all six subclasses.
High Cohesion	Each service (AppointmentService, NotificationService, AIRecommendationEngine) has one clear purpose.
Low Coupling	Presentation, Application and Domain packages depend only downward (Section 10); the AI module is isolated behind a stable interface.
Design by Contract	Pre/postconditions defined per use case (Section 3) constrain state transitions in the state machine (Section 9).

14.2 Design Patterns Selected and Justified

Pattern	Type	Where Applied	Justification
Strategy	Behavioural	AIRecommendationEngine ranking algorithm	Allows the ranking heuristic/ML model to be swapped (rule-based vs. ML-based) without changing AppointmentService callers.

Observer	Behavioural	Notification dispatch on Appointment state change	Patient, reminder scheduler and analytics module all react to Appointment status changes without the Appointment class knowing about its subscribers.
Factory Method	Creational	Creating role-specific User subclass instances during registration/admin user-creation	Centralises and hides the logic for choosing which User subclass to instantiate based on role.
Facade	Structural	AppointmentService as a single entry point over Slot, Doctor, AIRecommendationEngine, Notification	Simplifies the Presentation layer's interaction with a complex booking subsystem (supports Low Coupling).
State	Behavioural	Appointment lifecycle (Requested/Confirmed/InProgress/Completed/Cancelled/No-Show)	Encapsulates state-specific behaviour/transition rules directly, matching the state diagram in Section 9 and avoiding large conditional blocks.
Repository	Architectural	Persistence package (Section 10)	Decouples domain classes from the underlying data store, supporting maintainability and testability.

15. Assumptions and Constraints

15.1 Assumptions

- Each patient, doctor and staff member has exactly one active account (role) at a time.
- A consultation slot belongs to exactly one doctor and can be booked by at most one patient at a time.
- The AI recommendation engine is advisory; the final choice of doctor/slot always rests with the patient or reception staff.
- Historical appointment data required for AI ranking is available and anonymised/aggregated for the algorithm's use, complying with data-privacy policy.
- Payment processing (where fees apply) is delegated to an external payment gateway and is out of scope for detailed modelling.
- Laboratory and pharmacy modules operate on report/prescription identifiers already linked to a valid appointment; they do not create new appointments.

15.2 Constraints

- The system boundary, actors, and all classes/attributes/operations must be traceable back to the problem statement (Section 16).

- All diagrams must remain mutually consistent: operations invoked in Sequence/Activity diagrams must exist on the corresponding class in the Class Diagram.
- Role-based access control must be enforced at the service layer for every clinical and prescription-status operation.
- The platform must scale horizontally (stateless application-server nodes) to meet peak outpatient-clinic demand.
- Regulatory constraints (data protection / medical-record retention) govern storage and access to ClinicalNote, Prescription and LabReport data.

16. Requirement-to-Use Case/Class Traceability

This matrix demonstrates that every functional requirement (Section 1.3) is realised by at least one use case (Section 3/4) and implemented by at least one class/operation (Section 5/6), satisfying the completeness and traceability constraint.

Req. ID	Use Case(s)	Realising Class(es) / Operation(s)
FR-01	UC-01 Register/Manage Profile	User.updateProfile(), Patient (attributes)
FR-02	UC-02 Search Doctor	Patient.searchDoctor(), Doctor (attributes)
FR-03	UC-03, UC-04 Book/Reschedule/Cancel	Appointment.confirm()/reschedule()/cancel(), Slot.book()/release()
FR-04	UC-05 Receive Reminders	Notification.send(), NotificationService
FR-05	UC-06 View Prescription	Patient.viewPrescription(), Prescription
FR-06	UC-08 Submit Service Request	Patient.submitServiceRequest(), ServiceRequest.submit()
FR-07	UC-09 Manage Availability	Doctor.setAvailability(), Slot
FR-08	UC-10, UC-11, UC-12	Doctor.addClinicalNote()/issuePrescription()/requestFollowUp()
FR-09	UC-13 AI Recommendation	AIRecommendationEngine.recommendDoctor()/recommendSlot()
FR-10	UC-14 Update Lab Report Status	LabStaff.updateReportStatus(), LabReport.updateStatus()
FR-11	UC-15 Update Prescription Status	PharmacyStaff.updatePrescriptionStatus(), Prescription.updateStatus()
FR-12	UC-03/UC-04 (Reception path)	ReceptionStaff.bookAppointment(), Appointment
FR-13	UC-16, UC-17	Administrator.manageUser()/generateReport()

17. Implementation / Prototype and Results

A minimal working prototype is recommended to validate the model before full development. The suggested scope and stack for the prototype phase are summarised below; students should replace this section with their own implementation screenshots, test results and observations.

Aspect	Recommendation
Scope for prototype	UC-01 (Register), UC-02 (Search Doctor), UC-03 (Book Appointment incl. a simple rule-based stand-in for UC-13), UC-09 (Manage Availability).
Suggested stack	Frontend: React/Angular (Presentation package). Backend: Spring Boot / Node.js (Application + Domain packages). DB: PostgreSQL/MySQL (Persistence package). AI stub: a rule-based scoring function (specialty match + earliest slot) standing in for the ML model.
Expected results to record	Screenshots of doctor search & AI-ranked results, successful booking confirmation, an appointment status change, and a sample generated report/analytics view.

18. Reflection on Design Decisions and Learning Outcomes

18.1 Key Design Decisions and Rationale

- Modelling User as an abstract superclass rather than duplicating login/profile logic across six role classes reduced coupling and improved maintainability (supports CO2/CO3).
- Treating the AI Recommendation Engine as a separate «service» class with a Strategy-pattern-friendly interface keeps the intelligent-ranking logic replaceable without destabilising the core booking workflow.
- Using a State pattern for Appointment status directly reflects the lifecycle constraints in the problem statement and keeps the Class, Activity and State diagrams consistent with one another.
- A layered package structure (Presentation / Application / Domain / cross-cutting services) was chosen over a single monolithic package to satisfy the scalability and modularity constraints explicitly required by the assignment.

18.2 Learning Outcomes Addressed

CO	How this assignment addresses it
CO1	The problem was analysed using an object-oriented systems-development lifecycle: requirement summary -> actor/use-case identification -> object/class identification -> diagrammatic modelling.
CO2	All eight required UML diagram types (Use Case, Class, Sequence, Activity, State, Package, Component, Deployment) were produced and cross-checked for consistency.
CO3	Objects, classes, attributes, operations and relationships (association, aggregation, composition, generalisation, dependency) were explicitly identified and justified through use-case and CRC analysis.

18.3 Challenges and Future Improvements

- Balancing diagram detail against readability -- the Class Diagram was scoped to core domain classes; boundary/control classes are described in Section 5 rather than fully diagrammed, to keep the model legible.
- A production system would need a more detailed AI model design (feature set, training/retraining pipeline) beyond the Strategy-pattern interface shown here.
- Future iterations could add a Sequence Diagram for the lab-report and pharmacy-status update flows, and a more detailed security/audit sub-model.